

Aula 06 — Boas práticas: camadas, validação e tratamento de erros

Curso de Formação de Programação Aplicada · Módulo 1 · FPA Labs

Duração: 2h · **Pré-requisito:** Aula 05

Objetivos de aprendizagem

Ao final desta aula o aluno deve conseguir:

1. Explicar por que dividir o código em camadas e qual é a responsabilidade de cada uma.
2. Refatorar uma API de arquivo único para a estrutura routes → controller → service → repository.
3. Usar `express.Router()` para agrupar rotas.
4. Centralizar o tratamento de erros num único middleware.
5. Guardar configuração em variáveis de ambiente com um arquivo `.env`.

Plano de aula (2h)

Tempo	Bloco	Conteúdo
0:00–0:10	Aquecimento	Correção do exercício 6 da aula 05 (regra de negócio, 409)
0:10–0:30	Conceito	O problema do arquivo único · separação de responsabilidades
0:30–1:05	Código ao vivo	Refatorar a API de tarefas em camadas, passo a passo
1:05–1:15	Intervalo	
1:15–1:50	Mão na massa	Exercícios 1 a 3 (refatorar a API de filmes)
1:50–2:00	Fechamento	Erros customizados e <code>.env</code> · prévia da aula 07

Aviso ao professor: esta é a aula mais "abstrata" do módulo e a que mais gera a reação "*mas antes funcionava e era mais simples*". Ela **precisa** ser vendida com dor, não com teoria. Comece mostrando o `server.js` de 250 linhas e peça para acharem onde muda a regra de prioridade.

1. O problema

Seu `server.js` funciona. Mas ele tem 250 linhas e faz, ao mesmo tempo:

- interpretar HTTP (ler `req.body`, escolher status);
- validar dados;
- aplicar regras de negócio (não duplicar título, recalcular média);
- guardar e buscar dados.

Quatro responsabilidades diferentes, misturadas. Consequências reais:

Situação	O que acontece hoje
Trocar array por banco de dados	mexer em todas as rotas
Reaproveitar a regra "não duplicar" num script	impossível — ela está grudada no <code>res.status(409)</code>
Testar a validação sem subir servidor	impossível
Duas pessoas mexendo na API	conflito de merge garantido

Analogia do restaurante. Ninguém monta uma cozinha onde a mesma pessoa recebe o cliente, cozinha, guarda no estoque e lava a louça — ao mesmo tempo, na mesma bancada. Existe o **garçom** (fala com o cliente), o **cozinheiro** (sabe as receitas) e o **estoquista** (sabe onde estão os ingredientes). Se trocar o fornecedor, só o estoquista precisa saber. É exatamente isso que vamos fazer com o código.

2. As camadas

Requisição HTTP



ROUTES	"que rotas existem?"	→ só declara caminhos
CONTROLLER	"traduz HTTP ↔ negócio"	→ lê req, devolve res
SERVICE	"quais são as regras?"	→ o cérebro
REPOSITORY	"onde os dados ficam?"	→ só acessa dados



Array / Banco de dados

A regra que amarra tudo:

Camada	Sabe o que é HTTP?	Sabe onde os dados ficam?
Routes	✅ sim	❌ não
Controller	✅ sim	❌ não
Service	❌ não	❌ não
Repository	❌ não	✅ sim

Se você escrever `res.status(...)` dentro de um service, algo está errado. O service não sabe que existe HTTP — ele poderia ser chamado por um script de terminal, por um job agendado ou por um teste automatizado.

O que cada camada faz, na prática

Repository — só entrada e saída de dados. Nada de regra.

```
export const tarefaRepository = {
  listarTodas: () => tarefas,
  buscarPorId: (id) => tarefas.find((tarefa) => tarefa.id === id),
  criar: (dados) => { const nova = { id: proximoId++, ...dados }; tarefas.push(nova);
    return nova; },
  atualizar: (id, mudancas) => { /* ... */ },
  remover: (id) => { /* ... */ },
};
```

Service — as regras. Não sabe o que é `req`, `res` ou status code.

```

export const tarefaService = {
  criar(dados) {
    const erros = validarNovaTarefa(dados);
    if (erros.length > 0) throw new ErroDeValidacao(erros);

    if (tarefaRepository.existeComTitulo(dados.titulo)) {
      throw new ErroDeConflito("Já existe uma tarefa com esse título");
    }

    return tarefaRepository.criar({
      titulo: dados.titulo.trim(),
      concluida: false,
      prioridade: dados.prioridade ?? "media",
      criadaEm: new Date().toISOString(),
    });
  },
};

```

Controller — o tradutor: extrai da requisição, chama o service, devolve a resposta.

```

export const tarefaController = {
  criar(req, res, next) {
    try {
      const nova = tarefaService.criar(req.body);
      res.status(201).json(nova);
    } catch (erro) {
      next(erro); // manda para o middleware de erro
    }
  },
};

```

Routes — só o mapa.

```
const router = Router();

router.get("/", tarefaController.listar);
router.post("/", tarefaController.criar);
router.get("/:id", tarefaController.buscarPorId);
router.patch("/:id", tarefaController.atualizar);
router.delete("/:id", tarefaController.remover);

export default router;
```

Leia esse arquivo: em 5 linhas você entende a API inteira. **É esse o ganho.**

3. `express.Router()`

O `Router` é um mini-app que agrupa rotas relacionadas:

```
// routes/tarefa.routes.js
import { Router } from "express";
const router = Router();
router.get("/", ...);      // repare: só "/" – sem repetir /tarefas
export default router;

// app.js
app.use("/tarefas", tarefaRoutes); // o prefixo é definido aqui, uma vez só
```

Ganhos: o prefixo aparece em um lugar só, e adicionar um novo recurso (`/usuarios`) é criar um arquivo e uma linha no `app.js`.

4. Erros customizados

Como o service não conhece HTTP, ele não pode decidir o status. Mas ele **pode** dizer que tipo de problema aconteceu — criando classes de erro:

```
// erros/index.js
export class ErroDaAplicacao extends Error {
  constructor(mensagem, status) {
    super(mensagem);
    this.status = status;
  }
}

export class ErroNaoEncontrado extends ErroDaAplicacao {
  constructor(mensagem = "Recurso não encontrado") {
    super(mensagem, 404);
  }
}

export class ErroDeValidacao extends ErroDaAplicacao {
  constructor(erro) {
    super("Dados inválidos", 400);
    this.erro = erro;
  }
}

export class ErroDeConflito extends ErroDaAplicacao {
  constructor(mensagem) {
    super(mensagem, 409);
  }
}
```

Agora o service escreve `throw new ErroNaoEncontrado(...)` — uma frase que descreve o problema no vocabulário do negócio, sem mencionar HTTP. Quem traduz para status é a camada de cima.

5. Middleware de erro: um lugar só

O Express tem um tipo especial de middleware, com **quatro** parâmetros. Esse quarto parâmetro (`err` na primeira posição) é o que o marca como tratador de erros:

```
// middlewares/tratadorDeErros.js
export function tratadorDeErros(erro, req, res, next) {
  console.error(erro); // sempre registre – você vai precisar disso em produção

  if (erro instanceof ErroDeValidacao) {
    return res.status(400).json({ erros: erro.erros });
  }

  if (erro instanceof ErroDaAplicacao) {
    return res.status(erro.status).json({ erro: erro.message });
  }

  // Erro que não previmos: nunca exponha detalhes internos ao cliente.
  res.status(500).json({ erro: "Erro interno do servidor" });
}
```

Ele precisa ser registrado **por último**, depois de todas as rotas:

```
app.use("/tarefas", tarefaRoutes);
app.use(rotaNaoEncontrada); // 404
app.use(tratadorDeErros); // sempre o último
```

Com isso, cada controller vira três linhas: `try`, chama o service, `catch (erro) { next(erro) }`. Todo o tratamento vive num lugar só.

⚠ **Pegadinha:** em código **assíncrono**, um `throw` fora do `try/catch` não chega no middleware de erro. Por isso todo controller que usa `await` precisa do `try/catch` com `next(erro)`. Você vai sentir isso na aula 07, quando o repository virar assíncrono.

6. Variáveis de ambiente

Porta, senha de banco, chave de API — nada disso deveria estar escrito no código. Motivos:

1. **Muda por ambiente.** Sua máquina, homologação e produção têm configurações diferentes.
2. **Segredo não vai para o Git.** Chave commitada é chave vazada.

O Node lê arquivos `.env` nativamente:

```
# .env - NUNCA vai para o Git
PORTA=3000
NODE_ENV=desenvolvimento
```

```
"scripts": {
  "dev": "node --watch --env-file=.env src/server.js"
}
```

```
const PORTA = process.env.PORTA ?? 3000; // sempre tenha um padrão
```

E **sempre** crie um `.env.example` — este sim versionado — com as chaves e sem os valores:

```
# .env.example - este VAI para o Git
PORTA=
NODE_ENV=
```

Assim quem clonar o projeto sabe o que precisa configurar. Adicione `.env` ao `.gitignore` **no mesmo minuto** em que criar o arquivo. Nunca depois.

7. A estrutura final

```
api-tarefas/
├── .env                ← não vai para o Git
├── .env.example       ← vai
├── .gitignore
├── package.json
└── src/
    ├── server.js      ← só sobe o servidor
    ├── app.js         ← monta o Express e os middlewares
    ├── routes/
    │   └── tarefa.routes.js
    ├── controllers/
    │   └── tarefa.controller.js
    ├── services/
    │   └── tarefa.service.js
    ├── repositories/
    │   └── tarefa.repository.js
    ├── validacoes/
    │   └── tarefa.validacao.js
    ├── middlewares/
    │   ├── tratadorDeErros.js
    │   └── rotaNaoEncontrada.js
    └── erros/
        └── index.js
```

Note `server.js` e `app.js` separados: `app.js` monta a aplicação, `server.js` a coloca no ar. Parece exagero agora, mas é o que permite escrever testes automatizados sem subir uma porta.

Sim, são mais arquivos. Mas cada um é pequeno, tem um nome que diz o que ele faz, e você abre exatamente o que precisa. Compare: "onde está a regra de duplicidade?" → `services/`. Sempre.

8. A prova de que valeu a pena

Na aula 07 vamos trocar o array em memória por um banco SQLite. Com o código de hoje, **só o repository muda**. Controllers, services, rotas e validações ficam intactos.

Guarde esta frase para a semana que vem: *"lembram que eu disse que só um arquivo ia mudar?"*.

9. Ferramentas novas desta aula

`class`, `extends` e `super`

Esta é a única vez no módulo em que usamos classes. Não se assuste — a ideia é simples.

Uma **classe** é um molde para criar objetos parecidos:

```
class Pessoa {  
  constructor(nome) {    // roda automaticamente na criação  
    this.nome = nome;    // "this" é o objeto que está sendo criado  
  }  
}  
  
const ana = new Pessoa("Ana"); // "new" significa "crie um a partir deste molde"  
ana.nome;                      // "Ana"
```

`extends` faz uma classe herdar tudo de outra:

```
class ErroNaoEncontrado extends ErroDaAplicacao {  
  constructor(mensagem) {  
    super(mensagem, 404); // chama o constructor da classe "pai"  
  }  
}
```

- `extends X` → "aproveite tudo que `X` já tem"
- `super(...)` → "pai, faça a sua parte primeiro"

Por que herdamos de `Error`? Porque `Error` é o tipo nativo de erro do JavaScript. Herdando dele, nossos erros funcionam com `throw`, são capturados por `catch`, e já vêm com `.message` e `.stack` prontos — sem precisarmos escrever nada disso.

`instanceof`

O que faz: responde se um objeto foi criado a partir de determinada classe.

```
erro instanceof ErroDeValidacao; // true ou false
```

É assim que o tratador de erros decide o que fazer:

```
if (erro instanceof ErroDeValidacao) return res.status(400).json({ erros: erro.erros });  
if (erro instanceof ErroDaAplicacao) return res.status(erro.status).json({ erro: erro.message });  
res.status(500).json({ erro: "Erro interno do servidor" }); // não é nenhum dos nossos → bug
```

A ordem importa: como `ErroDeValidacao` **também** é um `ErroDaAplicacao` (por causa do `extends`), o teste mais específico precisa vir primeiro.

`next()` — passando a bola

Todo middleware recebe um terceiro parâmetro, `next`. Ele é uma **função**:

```
app.use((req, res, next) => {
  console.log(req.method);
  next(); // ⚠ sem isto, a requisição para aqui e nunca chega na rota
});
```

Como você chama	O que acontece
<code>next()</code>	segue para o próximo middleware ou rota
<code>next(erro)</code>	pula direto para o tratador de erros
não chamar	a requisição trava e o cliente fica esperando

`next(erro)` é o que faz o `catch` de cada controller funcionar: em vez de responder ali, ele entrega o problema para quem sabe traduzi-lo em status HTTP.

`res.on("finish", callback)`

O que faz: executa uma função quando a resposta **terminou** de ser enviada. **Quando usar:** registrar log com o status final e o tempo de resposta.

```
res.on("finish", () => {
  console.log(`${req.method} ${req.originalUrl} → ${res.statusCode}`);
});
```

Por que não registrar antes? Porque no começo da requisição o status ainda não existe — ele só é decidido lá na frente, pela rota ou pelo tratador de erros.

`readFileSync` e `import.meta.url`

```
import { readFileSync } from "node:fs";

const schema = readFileSync(new URL("./schema.sql", import.meta.url), "utf-8");
```

- `readFileSync` é a versão **síncrona** do `readFile`: ela bloqueia até terminar. Usamos só na **inicialização** do programa, onde bloquear por 2 ms não faz diferença. Nunca use dentro de uma rota.

- `import.meta.url` é o endereço do arquivo atual. Combinado com `new URL(...)`, garante que o caminho funcione independentemente da pasta de onde você rodou o `node`.
-

Recapitulação em 3 frases

1. Cada camada tem uma responsabilidade: rota declara, controller traduz HTTP, service decide, repository guarda.
2. Service nunca vê `req / res`; ele lança erros de negócio e o middleware traduz para status.
3. Configuração vive no `.env`, que nunca vai para o Git — o `.env.example` vai no lugar dele.

Tarefa para a próxima aula

Exercícios 1 a 3 de [exercicios/README.md](#): refatore sua API de filmes. **Essa refatoração é pré-requisito para a aula 07** — chegue com ela pronta.

Na próxima aula

Banco de dados. Seus dados finalmente vão sobreviver ao restart do servidor.
